INTRODUCCIÓN A LA PROGRAMACIÓN · 1.º GII · TEORÍA

Guía de teoría ← ≠ ≤

Los temas 0 a 4 resumidos: definiciones, la notación algorítmica (SP) con su traducción a C++, los problemas de clase resueltos y las trampas típicas. En el tema 4 tienes un simulador de la máquina secuencial y al final, las chuletas.

Tema 0

0 Algoritmos, programas y procesos

¿Qué tareas puede realizar un ordenador? Los conceptos de base: computador, algoritmo, lenguaje, programa, proceso, estado y variable.

0.1 Computador y algoritmo

Un **computador** es un **mecanismo digital de propósito general** que se convierte en un mecanismo para un **uso específico** cuando procesa un determinado algoritmo. El mismo hardware sirve para todo: lo que lo especializa en cada momento es el algoritmo que ejecuta.

Un **algoritmo** es una **secuencia de instrucciones** (operaciones) que especifican las acciones que debe ejecutar un computador para realizar una tarea.

Área de un triángulo rectángulo (en lenguaje natural)

```
INICIO
  Obtener las medidas de la base (b)
y la altura (h)
  Multiplicar base por altura (b ×
h)
  Dividir entre 2 el resultado (b ×
h) / 2
FIN
```

El mismo estilo en notación algorítmica

```
Léxico
  a, b, c: Entero;
Algoritmo
  b ← 2;
  a ← 3;
  c ← a + b;
  Escribir (c);
Fin.
```

La noción de algoritmo es **más general** que la informática: cualquier receta que resuelve un problema lo es.

Algoritmo	Problema (tarea)
Instrucciones para tejer un jersey	Tejer un jersey
Partitura	Tocar una sonata
Plan de trabajo	Cursar una licenciatura
Receta de cocina	Cocinar un bizcocho
Método de Newton	Calcular las raíces

0.2 Lenguajes de programación y traductores

El computador sólo ejecuta **instrucciones máquina**, muy simples. Un computador está caracterizado por el conjunto de instrucciones que puede ejecutar. Formato: **código + [operandos]** (entre 0 y 3). El código indica la operación (suma, resta, comparación, salto...) y los operandos, direcciones de memoria o valores.

Los **lenguajes de programación** son notaciones que expresan las operaciones que queremos que haga el ordenador con un **nivel de abstracción superior** al del lenguaje máquina. Por eso hace falta un **traductor**: **compiladores** (traducen el programa completo a código máquina antes de ejecutarlo; es lo que hace `g++` en prácticas) e **intérpretes** (traducen y ejecutan instrucción a instrucción).



Del algoritmo al proceso. El compilador traduce el programa entero antes de ejecutarlo; el intérprete traduce y ejecuta instrucción a instrucción.

- El **mismo algoritmo** se puede expresar en **lenguajes diferentes** (C++, C, Python...).
- En el análisis se usa una **notación algorítmica** (la de clase, que llamaremos **SP**) para evitar los detalles del lenguaje y centrarse en el proceso que debe llevar a cabo el computador.
- Un **programa** es la expresión de un algoritmo en un lenguaje de programación concreto.

Algoritmo

```
Léxico
  numero: Entero;
Algoritmo
  Escribir ('Este es el segundo programa');
  Escribir ('Escribe un número');
  Leer (numero);
  Escribir ('La variable numero ha tomado el valor ', numero);
Fin.
```

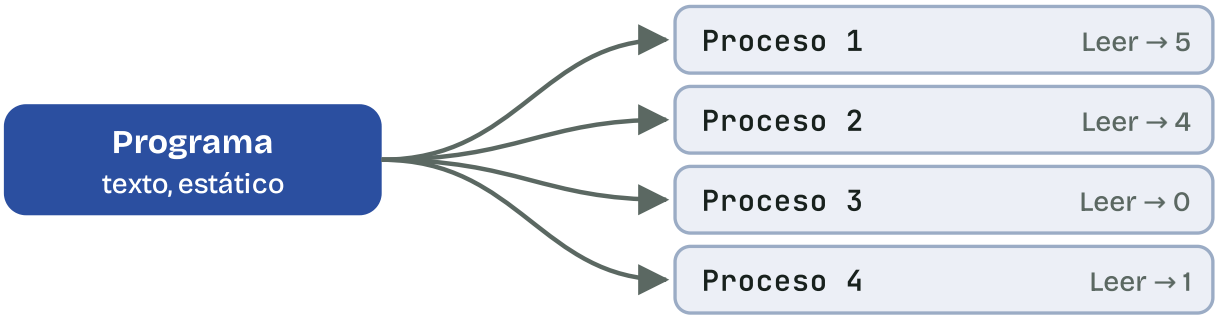
Programa (C++)

```
#include <iostream>
using namespace std;
int main ( ) {
    int numero;
    cout << "Este es el segundo programa" << endl;
    cout << "Escribe un número ";
    cin >> numero;
    cout << "La variable numero ha tomado el valor " << numero << endl;
    return 0;
}
```

0.3 Programa, proceso, estado y variable

- **Proceso**: un **programa en ejecución**. Se caracteriza por una **sucesión de estados**, desde un estado inicial hasta un estado final.
- **Estado** de un proceso: determina su grado de progreso; viene dado por los **valores de las variables** y un **indicador de la siguiente instrucción** a ejecutar.
- **Variable**: magnitud o información manejada por un algoritmo a la que se da un **nombre** y que **registra un valor**. Representa una entidad del mundo real relevante para el problema (la altura, una nota...).

Si ejecutamos el programa anterior cuatro veces con 5, 4, 0 y 1, tenemos **un programa y cuatro procesos**: el texto es el mismo, pero cada ejecución recorre sus propios estados.



Un programa y cuatro procesos: el texto es el mismo, pero cada ejecución recorre sus propios estados.

> ⚠ **Ojo en el test**
>
> Programa ≠ proceso. El programa es estático (texto); el proceso es dinámico (ejecución). Y el estado no son sólo las variables: también cuenta por dónde va la ejecución.

0.4 ¿Qué tareas puede realizar un ordenador?

Clase	Definición
Computable	Existe un algoritmo que lo resuelve.
· Tratable	Computable y existe un algoritmo **eficiente** (de orden polinomial) que lo resuelve.
· Intratable	Computable, pero no se conoce un algoritmo eficiente: los que hay crecen demasiado deprisa con el tamaño del problema.
No computable	No existe ningún algoritmo que lo resuelva.

Tema 1

1 Léxico y organización de un algoritmo

Qué es el léxico de un algoritmo, los tipos primitivos con sus operaciones, las acciones primitivas y las dos primeras formas de organizar las acciones: secuencia y análisis de casos.

1.1 Léxico y control de un algoritmo

Un **algoritmo** = **léxico** (informaciones + acciones) + **control** (el orden y las condiciones en que se ejecutan las acciones: **secuenciación**, **análisis de casos** e **iteración** o recursión). Construir un algoritmo es elegir informaciones y acciones y decidir cómo organizarlas en el tiempo para obtener el resultado por acumulación de sus efectos.

Ejemplo, ir al cajero: *informaciones*: cantidad a reintegrar, cuenta, saldo; *acciones*: seleccionar cuenta, consultar saldo, reintegrar.

Abstracción: «eliminar detalles innecesarios y considerar lo esencial». Es el mecanismo fundamental para dominar la complejidad; el léxico (variables y acciones) fija el nivel de abstracción.

Elemento del léxico	Qué se especifica
Información (variable)	**nombre** y **tipo**
Acción	**nombre**, **precondición** (lo que debe cumplirse antes) y **postcondición** (lo que se cumple después)
Tipo de dato	**dominio** de valores y **operaciones** posibles sobre ellos

Ejemplo: para `retirar (cuenta, cantRet)`, *pre*: 0 < cantRet ≤ saldo; *post*: el nuevo saldo es el anterior menos cantRet.

Estructura de un algoritmo

```
LÉXICO
  // declaraciones de tipos, variables, constantes y acciones
  v1 : tipo;
  A1 : una acción
    PRE { precondición de A1 }
    POST { postcondición de A1 }
    LÉXICO  … ALGORITMO … FIN ALGORITMO
  PRE { precondición del algoritmo }
  POST { postcondición del algoritmo }
  // secuencia de instrucciones
FIN
```

Ejemplo

```
LÉXICO
  tempFahrenheit : Real;    // dato
  tempCelsius : Real;       // resultado
  ConvertirFahrenheitCelsius : una acción
    PRE { 0 ≤ tempFahrenheit ≤ 200 }
    POST { tempCelsius es tempFahrenheit en grados Celsius }
    ALGORITMO
      tempCelsius ← (5.0 / 9.0) * (tempFahrenheit - 32.0)
    FIN
ALGORITMO
  Leer (tempFahrenheit);
  ConvertirFahrenheitCelsius;
  Escribir ("La temperatura en grados Celsius es:", tempCelsius)
FIN
```

1.2 Tipos de datos primitivos

Nuestra notación incluye **Entero**, **Real**, **Booleano**, **Carácter** e intervalos (`i, j: [1, 100]`), además de mecanismos para definir tipos nuevos. De cada tipo hay que conocer el dominio, las operaciones, la sintaxis de los literales y la de las expresiones.

Tipo	Literales	Operaciones
Entero	`0, 352, -342`	`-` (cambio de signo); `+ - * DIV MOD`: Entero×Entero→Entero; `/`: **Entero×Entero→Real**; `< > = ≠ ≤ ≥` →Booleano; Predecesor, Sucesor
Real	`4.22, -23.44` (punto decimal)	`- + - * /`; comparaciones → Booleano
Booleano	`Verdadero, Falso`	`Y, O`: Booleano×Booleano→Booleano; `NO`
Carácter	`'A', 'a', '$', '1'` (entre apóstrofos)	`Car`: Entero→Carácter; `Ord`: Carácter→Entero; comparaciones; Predecesor, Sucesor

Con `númeroAlumnos = 200, cursos = 4, media = 5.0, nota = 6.0, peso = 80.0, letra = 'A', aprobado = Verdadero`:

Expresión	Tipo	Valor
`númeroAlumnos DIV cursos`	Entero	50
`númeroAlumnos MOD cursos`	Entero	0
`1000 - númeroAlumnos * 2`	Entero	600
`nota * peso / 100.0 - 3.0`	Real	1.8
`nota * (peso / 100.0 - 3.0)`	Real	−13.2
`nota > 7.0`	Booleano	Falso
`(númeroAlumnos DIV cursos) > 20`	Booleano	Verdadero
`(nota > 7.0) Y (media = 5.0)`	Booleano	Falso
`NO (letra = 'B') O aprobado`	Booleano	Verdadero

Expresión	Tipo	Valor
`Carácter (66)` · `Sucesor (letra)`	Carácter	'B' · 'B'

> ⚠ **Ojo: notación ≠ C++**
>
> En la notación, `7 / 2` = 3.5 (Real) y el cociente entero es `7 DIV 2` = 3. En C++, `7 / 2` con dos `int` da 3. Lo verás en la práctica 1.

1.3 Acciones primitivas: asignación, Leer y Escribir

La **asignación** `<variable> ← <expresión>` es la acción primitiva que caracteriza a los lenguajes imperativos: se **evalúa la expresión** de la derecha y el resultado se guarda en la variable de la izquierda (el valor anterior se pierde). El nuevo valor puede depender del anterior: `longitud ← longitud / 2`.

Entrada: `Leer (lista de variables)` asigna a cada variable el valor introducido por teclado. **Salida**: `Escribir (lista de expresiones)` envía al terminal los valores de evaluar cada expresión.

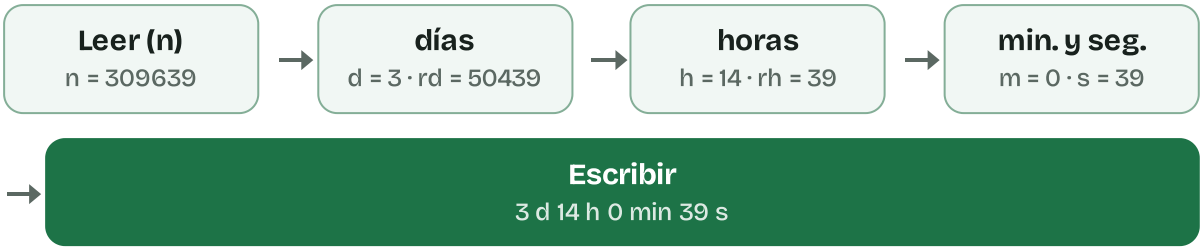
Leer y Escribir

```
LÉXICO
  nota : Real; nombre : Secuencia de caracteres; numexam : Entero;
ALGORITMO
  Leer (nota, nombre, numexam);       // teclea: 6.0 Ana 200
  Escribir (nota, nombre, numexam)    // pantalla: 6 Ana 200
FIN
```

1.4 Composición secuencial

Para reducir la complejidad se introducen **estados intermedios**: el problema se descompone en subproblemas más simples que se resuelven de forma independiente. `{E0} A1 {E1} A2 {E2} … {En}`.

Problema: dado un número de segundos $n < 10^6$, expresarlo en días, horas, minutos y segundos. Diseño: `n = 86400·d + 3600·h + 60·m + s`, con $0 \le h \le 23$ y $0 \le m, s \le 59$.



Cada acción lleva de un estado intermedio al siguiente: {E0} A1 {E1} A2 {E2} … Así se resuelve cada trozo por separado.

Segundos → días, horas, minutos y segundos

```
LÉXICO
  n : Entero [0, 999999];    // dato: nº de segundos
  d : Entero ≥ 0;            // días
  h : Entero [0, 23];        // horas
  m, s : Entero [0, 59];     // minutos y segundos
  rd : Entero [0, 86399];    // resto de días
  rh : Entero [0, 3599];     // resto de horas
ALGORITMO
  Leer (n);
  d ← n DIV 86400;  rd ← n MOD 86400;    { n = 86400·d + rd, 0 ≤ rd ≤ 86399 }
  h ← rd DIV 3600;  rh ← rd MOD 3600;    { rd = 3600·h + rh, 0 ≤ rh ≤ 3599 }
  m ← rh DIV 60;    s ← rh MOD 60;       { n = 86400·d + 3600·h + 60·m + s }
  Escribir (d, h, m, s)
FIN
```

Ejemplo resuelto

1. d = 309639 DIV 86400 = **3**; rd = 309639 − 259200 = 50439.
2. h = 50439 DIV 3600 = **14**; rh = 50439 − 50400 = 39.
3. m = 39 DIV 60 = **0**; s = 39 MOD 60 = **39**.

RESULTADO 309639 s = 3 días, 14 h, 0 min, 39 s

Es la misma estrategia del desglose de billetes de la práctica 2: cociente con el más grande y seguir con el resto.

1.5 Análisis de casos

Técnica de descomposición que **particiona el dominio** en subdominios (casos); cada subproblema es el problema restringido a su subdominio. La partición puede estar **guiada por los datos** (porcentaje de impuesto según la renta: < 12 000, 12 000–35 000, > 35 000) o **por los resultados** (sobresaliente [9, 10], notable [7, 9), aprobado [5, 7), suspenso [0, 5)).

suspenso	aprobado	notable	sobresal.
[0, 5)	[5, 7)	[7, 9)	[9, 10]

Partición guiada por los resultados: cada nota de [0, 10] cae en un caso y sólo en uno. Los casos cubren todo el dominio y no se solapan.

⚠ Condiciones de corrección

1. **Cubrir** todo el dominio inicial.
2. Que los subdominios **no se solapen**.

Además, la postcondición de cada caso debe cumplir la del problema.

Errores típicos: olvidar casos e incluir casos repetidos

```
// DESCOMPOSICIÓN INCORRECTA (edad en [0, 120])
if (edad ⩾ 0 && edad ⩽ 12) {          // Infantil
} else if (edad ⩾ 12 && edad ⩽ 18) {  // Joven: SOLAPAMIENTO en 12
} else if (edad > 20) {               // Adulto: FALTA COBERTURA de 19 y 20
}
```

Composiciones de la notación:

SEGÚN

```
SEGÚN c1, c2, …, cn
  e1 : a1;
  e2 : a2;
  …
  en : an;
  EN_OTRO_CASO : an+1    // si ninguna ei
FIN_SEGÚN
```

SI como caso particular

```
SI e ENTONCES a SI_NO b FIN_SI
  ≡ SEGÚN …  e : a;  NO (e) : b
FIN_SEGÚN

SI e ENTONCES a FIN_SI
  ≡ SEGÚN …  e : a;  NO (e) : ;
FIN_SEGÚN    // acción vacía
```

Máximo de dos enteros. Especificación: pre {x = X, y = Y}; post {z = máx (X, Y)}. La composición secuencial no permite decidir según los datos: hace falta la alternativa.

Máximo de dos enteros

```
ALGORITMO
  PRE { x, y : Entero; x = X, y = Y }
  Leer (x, y);
  SEGÚN x, y
    x ≥ y : z ← x;
    x < y : z ← y;
  FIN_SEGÚN;
  Escribir (z)
  POST { x = X, y = Y, ((z = X) O (z = Y)) Y (z ≥ X) Y (z ≥ Y) }
FIN
```

Ordenar tres enteros distintos (pre: X ≠ Y ≠ Z ≠ X; post: (p, s, t) ∈ perm (X, Y, Z) y p < s < t). El resultado es una de las **3! = 6** permutaciones.

Solución 1: los seis casos (guiada por los datos)

```
Leer (a, b, c);
SEGÚN a, b, c
  a<b<c : p ← a; s ← b; t ← c;
  a<c<b : p ← a; s ← c; t ← b;
  b<a<c : p ← b; s ← a; t ← c;
  b<c<a : p ← b; s ← c; t ← a;
  c<b<a : p ← c; s ← b; t ← a;
  c<a<b : p ← c; s ← a; t ← b;
FIN_SEGÚN;
Escribir (p, s, t)
```

Solución 3: guiada por los resultados

```
p ← a; s ← b; t ← c;
SEGÚN p, s, t
  (p < s) Y (p < t) : ;
// acción vacía
  (s < p) Y (s < t) : Intercambio
(p, s);
  (t < p) Y (t < s) : Intercambio
(p, t);
FIN_SEGÚN;
SI s > t ENTONCES Intercambio (s, t)
FIN_SI;
Escribir (p, s, t)
```

La solución 2 anida dos SEGÚN: primero compara a y b y después sitúa c (tres casos en cada rama).

1.6 Tipos no primitivos: producto de tipos (registro)

Los valores de un tipo pueden ser **atómicos** (escalares) o **estructurados**. Los constructores habituales son la **tabla** (array), la **estructura** (producto de tipos) y la **secuencia**.

Producto de tipos (registro, estructura): sus valores son agregaciones de valores de otros tipos ya definidos. `nombre = TIPO < a1: T1; a2: T2; …; aN: TN >`. Dominio: las **N-tuplas**; cardinalidad: $C = C_{T1} \cdot C_{T2} \cdot \ldots \cdot C_{TN}$.

Notación SP

```
LÉXICO
  Estudiante = TIPO < nombre :
secuencia de Carácter;
                      edad : Entero;
                      sexo :
Booleano;
                      centro :
Entero [1, 20];
                      curso : Entero
[1, 6] >;
  a1 : Estudiante;
// a1 = <"J. Gómez", 21, F, 4, 3>
// acceso: a1.curso (o «curso de
a1»)
```

C++ (práctica 4)

```
struct Estudiante {
    string nombre;
    int edad;
    bool sexo;
    int centro;     // 1..20
    int curso;      // 1..6
};
Estudiante a1;
a1.curso = 3;
```

Tema 2

2 Acciones, funciones y tablas

Parametrización de acciones (dato, resultado, dato–resultado), funciones y composición funcional, ámbito de los identificadores y el tipo tabla.

2.1 Acciones parametrizadas

Una **acción** (procedimiento) describe una acción compleja en función de otras más simples y se usa igual que una primitiva. Su algoritmo combina acciones para modificar el estado del proceso y lograr la postcondición a partir de la precondición.

Motivación: la acción `AñoBisiesto` sin parámetros sólo sirve para la variable global `f`. Si tenemos `hoy, mañana, ayer: Fecha`, ¿cómo la usamos? **Parametrizando**: «mayor parametrización supone mayor independencia del contexto: más reutilizable».

Acción con un parámetro dato

```
AñoBisiesto : acción (dato f :
Fecha)
  POST { esBisiesto es Verdadero si
f.año es bisiesto }
  ALGORITMO
    esBisiesto ← ((f.año MOD 4 = 0)
Y (f.año MOD 100 ≠ 0))
                 O (f.año MOD 400 =
0)
  FIN
…
AñoBisiesto (hoy);
AñoBisiesto (mañana);
```

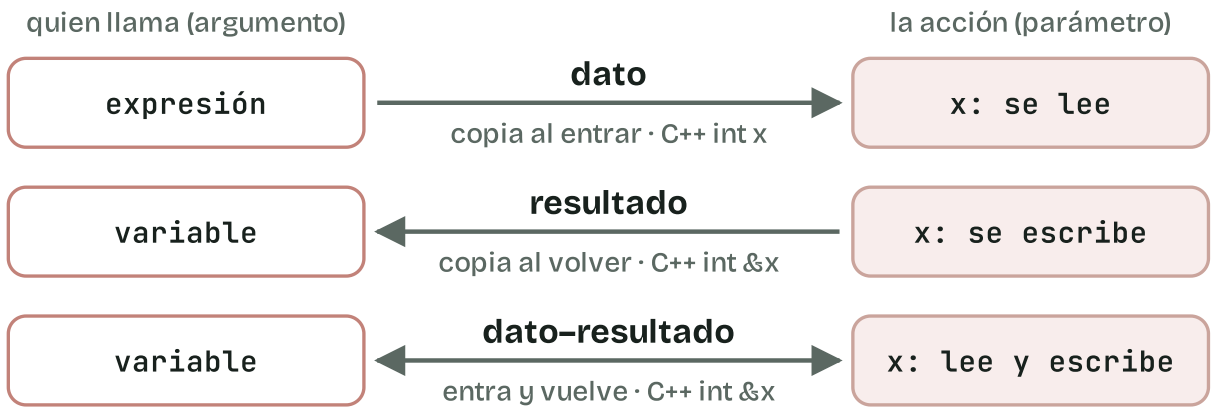
Intercambio

```
LÉXICO
  a, b : Entero;
  Intercambio : acción (dato-
resultado x, y : Entero)
    LÉXICO
      t : Entero;           //
variable local
    ALGORITMO
      t ← x; x ← y; y ← t
    FIN;
ALGORITMO
  Leer (a, b);              { a = A,
b = B }
  Intercambio (a, b);       { a = B,
b = A }
  Escribir (a, b)
FIN.
```

Notación: `nombre : acción (clase1 p1 : tipo1; …; clasen pn : tipon)`, donde la clase es **dato**, **resultado** o **dato–resultado**. En la definición están los **parámetros formales**; en la llamada, los **argumentos** (parámetros reales).

2.2 Clases de parámetros

Clase	Papel	En la llamada / el retorno	Argumento	C++
dato	Entrada: se suministra y no debe modificarse	parámetro ← argumento	Cualquier **expresión** del tipo	por valor: `int x`
resultado	Salida: valor calculado por la acción	argumento ← parámetro (al volver)	Una **variable**	por referencia: `int &x`
dato–resultado	Entrada/salida: se usa su valor y se modifica	las dos cosas	Una **variable**	por referencia: `int &x`



Hacia dónde viaja el valor en cada clase de parámetro.

Al emparejar parámetros formales y argumentos deben coincidir en **número, tipo y posición**. Los identificadores **no** tienen que coincidir: los parámetros son parte del léxico local, no se ven fuera y pueden enmascarar nombres del léxico superior.

Valor decimal de un binario de cuatro cifras (versión corregida de las transparencias)

```
ValDec_NumBin : acción (dato bit3, bit2, bit1, bit0 : Entero [0, 1];
                        resultado valDec : Entero)
  ALGORITMO
    valDec ← bit3*8 + bit2*4 + bit1*2 + bit0
  FIN;
ALGORITMO
  x ← 0; y ← 1;
  ValDec_NumBin (1, 1, 1, 0, vD);   Escribir (vD);   // 14
  ValDec_NumBin (y, x, y, x, vD);   Escribir (vD);   // 1010₂ = 10
FIN.
```

Traza de la llamada

Al llamar `ValDec_NumBin (y, x, y, x, vD)`: bit3 ← 1, bit2 ← 0, bit1 ← 1, bit0 ← 0 (copias). Se calcula valDec = 8 + 2 = 10 y, **al volver**, vD ← valDec = 10. Con `Intercambio (a, b)` y a = 7, b = 4: al entrar x = 7, y = 4; al salir x = 4, y = 7, que se copian a a y b.

2.3 Abstracción y descomposición con acciones

Abstracción	Pregunta	Ejemplo
Por **especificación**	¿Qué hace? (frente a ¿cómo lo hace?)	Basta saber {x = X, y = Y} Intercambio {x = Y, y = X}
Por **parametrización**	Generalidad frente a especificidad	Intercambia dos variables enteras *cualesquiera*, no sólo «x» e «y»

Desplazamiento circular de a, b, c: {a = A, b = B, c = C} → {a = B, b = C, c = A}. Con un estado intermedio y la acción Intercambio:

Dos llamadas resuelven el problema

```
Leer (a, b, c);          { a = A, b = B, c = C }    // 12 80 25
Intercambio (a, b);      { a = B, b = A, c = C }    // 80 12 25
Intercambio (b, c);      { a = B, b = C, c = A }    // 80 25 12
Escribir (a, b, c)
```

Definir una acción nueva **enriquece** el conjunto de acciones disponibles. Las acciones son la base de la **descomposición de programas** en el paradigma imperativo, y gracias a la **localidad** se pueden escribir y entender sin conocer el contexto donde se usan: basta su especificación.

2.4 Funciones

	Acción	Función
Especificación	Modifica el estado del proceso (estado inicial → final)	Relación entre dominio y codominio: `f : A1 × … × An → B`
Uso	Como una acción primitiva (instrucción)	Como un **operador**, dentro de expresiones
Algoritmo	Combina acciones para lograr la postcondición	Calcula el valor del codominio que corresponde a los parámetros
Parámetros	dato, resultado o dato–resultado	**Todos dato**
Resultado	Por los parámetros	Un único valor, asignado al **nombre de la función** como último paso

Max2 sin análisis de casos

```
Max2 : función (x, y : Entero) → Entero;
  ALGORITMO
    Max2 ← ((x + y) + abs (x - y)) DIV 2
  FIN;
// Max2 (8, 3) = (11 + 5) DIV 2 = 8
```

Max3 por composición funcional

```
Max3 : función (x, y, z : Entero) → Entero;
  ALGORITMO
    Max3 ← Max2 (x, Max2 (y, z))
  FIN;
// Max3 (6, 3, 5) = Max2 (6, Max2 (3, 5)) = Max2 (6, 5) = 6
t ← a + b + c - Max3 (a, b, c);
SI Max3 (a, b, c) = a ENTONCES a ← a + 1 FIN_SI
```

Max3 se puede resolver de cuatro formas: (1) **composición funcional** con Max2; (2) **análisis exhaustivo** de casos (x mayor que y y que z...); (3) composición funcional con análisis de casos (si x > y, Max2 (x, z); si no, Max2 (y, z)); (4) análisis de casos **en dos etapas** (SEGÚN x, y y dentro SEGÚN con z). La 1 es la más legible y reutiliza lo ya hecho.

2.5 Ejemplo: suma de duraciones

Dadas dos duraciones <h, m, s>, obtener su suma expresada igual. Idea: pasar las dos a segundos, sumarlas y volver a pasar a duración.

SumarD

```
LÉXICO
  Duración = TIPO < h : Entero ≥ 0; m, s : Entero [0, 59] >;
  d1, d2 : Duración;
  SumarD : función (a, b : Duración) → Duración;
    LÉXICO
      Cvds : función (d : Duración) → Entero;       // duración → segundos
        ALGORITMO  Cvds ← 3600*d.h + 60*d.m + d.s  FIN;
      Cvsd : función (n : Entero) → Duración;       // segundos → duración
        LÉXICO  d : Duración; rh : Entero [0, 3599];
        ALGORITMO
          d.h ← n DIV 3600;  rh ← n MOD 3600;
          d.m ← rh DIV 60;   d.s ← rh MOD 60;
          Cvsd ← d
        FIN;
    ALGORITMO
      SumarD ← Cvsd (Cvds (a) + Cvds (b))
    FIN;
ALGORITMO
  Leer (d1, d2);
  Escribir (SumarD (d1, d2))
FIN
```

Comprobación

<3, 30, 59> = 12659 s y <1, 59, 5> = 7145 s. Suma 19804 s → h = 5, resto 1804 → m = 30, s = 4: **<5, 30, 4>**.

2.6 Ámbito y visibilidad

Una acción puede contener otras en su léxico. Cada identificador tiene **ámbito** en el bloque donde se declara y en los que éste contiene.

Léxico de	Tiene ámbito en
ALG (principal), que contiene A y B	ALG, A, A1, A2, B
A, que contiene A1 y A2	A, A1, A2
A1	A1
A2	A2
B	B

Regla de la máxima localidad: los identificadores de léxicos interiores **enmascaran** a los idénticos de niveles superiores. En C++ no se pueden definir funciones dentro de otras: todas van en el léxico global.

2.7 Tablas (arrays)

Una **tabla** asocia a cada valor de un **dominio** (un intervalo [bi, bs] de un tipo ordinal: los índices) un valor del **rango** (el tipo base, cualquiera). Número de elementos: `n = ordinal (bs) - ordinal (bi) + 1`. Se accede **directamente** a cada elemento por su índice: si `T: Temperatura`, T_2 es el elemento de índice 2.

Declaración y acceso

```
Notas = TIPO Tabla [1, 8] de Real;
Temperatura = TIPO Tabla [1, 12] de Real;
T : Temperatura;            // T = [10, 9.5, 11, 14.6, 17, 22, 27, 32, 27,
25, 16, 15]
media ← (T1 + T2) / 2       // media de enero y febrero = 9.75
```

Ejemplo: día del año de una fecha (día, mes). Si todos los meses tuvieran 30 días sería `30·(mes - 1) + día`; una tabla guarda la **corrección** acumulada de cada mes (enero tiene un día de más: +1 a partir de febrero; febrero tiene dos de menos: −1 a partir de marzo…).

Día del año con una tabla de correcciones

```
LÉXICO
  Día = TIPO Entero [1, 31];  Mes = TIPO Entero [1, 12];
  Fecha = TIPO < d : Día; m : Mes >;
  correc : Tabla [Mes] de Entero = [0, 1, -1, 0, 0, 1, 1, 2, 3, 3, 4, 4];
  x : Fecha;
ALGORITMO
  Leer (x);
  Escribir (30 * (x.m - 1) + x.d + correc[x.m])
FIN.
// (4, 2) → 30 + 4 + 1 = 35;  (30, 1) → 30;  (31, 12) → 330 + 31 + 4 = 365
```

Tabla	Producto de tipos
Elementos de idéntica naturaleza (mismo tipo)	Campos de tipos distintos
Acceso por **indexación** (T_i)	Acceso por **nombre** (a1.curso)
Nº de elementos significativos no fijo	Número fijo de campos

Operaciones frecuentes: **recorrido** (la misma acción sobre todos los elementos) y **búsqueda** (encontrar uno que cumpla una propiedad). Son los esquemas del tema

4.

Tema 3

3 La iteración y sus formas

Por qué hace falta iterar, las cuatro composiciones iterativas de la notación, sus equivalencias y los problemas clásicos resueltos.

3.1 La necesidad de iterar

Problema: calcular la media de las notas de un grupo cuyo tamaño no se conoce; se termina con una nota negativa. Con SI anidados habría que escribir el mismo bloque «sumar, contar, leer» tantas veces como alumnos… que no sabemos. Hace falta **repetir**.

Partes de toda iteración

- **Inicialización**: tratamientos previos.
- **Cuerpo**: las sentencias que se repiten.
- **Finalización**: lo que se hace cuando la iteración se detiene.

Y siempre debe tener un **punto de parada**.

3.2 MIENTRAS

`MIENTRAS cond HACER S FIN_MIENTRAS`: comprobación de parada **al principio**; la condición es de **continuación**. El cuerpo puede ejecutarse **cero** veces.

Media de notas con MIENTRAS

```
suma ← 0; numElem ← 0; Leer (nota);
MIENTRAS nota ≥ 0 HACER
  suma ← suma + nota; numElem ← numElem + 1;
  Leer (nota)
FIN_MIENTRAS;
SI numElem > 0 ENTONCES Escribir ("Valor medio de las notas: ", suma /
numElem)
SI_NO Escribir ("No hay notas que promediar")
FIN_SI
```

3.3 REPETIR ... HASTA

`REPETIR S HASTA cond`: parada **al final**; el cuerpo se ejecuta **al menos una vez**; la condición es de **parada** (al revés que en el MIENTRAS). Ideal cuando hay que hacer algo antes de poder comprobar: pedir un dato hasta que sea válido.

Validar una nota

```
REPETIR
  Escribir ("Ingrese la nota del alumno (0 a 10): ");
  Leer (nota);
  SI (nota < 0) O (nota > 10) ENTONCES
    Escribir ("Error: la nota debe estar entre 0 y 10.")
  FIN_SI
HASTA QUE (nota ≥ 0) Y (nota ≤ 10);
Escribir ("Nota registrada con éxito: ", nota)
```

3.4 ITERAR ... DETENER

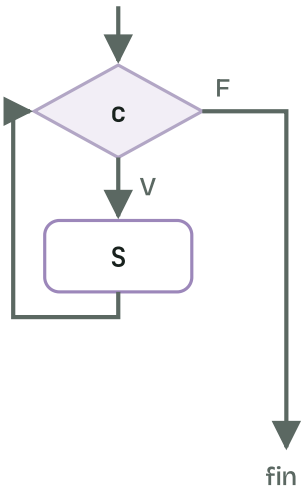
`ITERAR S2 DETENER cond S3 FIN_ITERAR`: parada **en medio**. S2 se ejecuta al menos una vez y S3 puede no ejecutarse. La condición es de **parada**. Encaja con «leer, comprobar si es el final, tratar».

Media de notas con ITERAR

```
suma ← 0; numElem ← 0;
ITERAR
  Leer (nota);
  DETENER nota < 0;
  suma ← suma + nota; numElem ← numElem + 1
FIN_ITERAR;
SI numElem > 0 ENTONCES Escribir ("Valor medio: ", suma / numElem)
SI_NO Escribir ("No hay notas que promediar") FIN_SI
```

3.5 Equivalencias

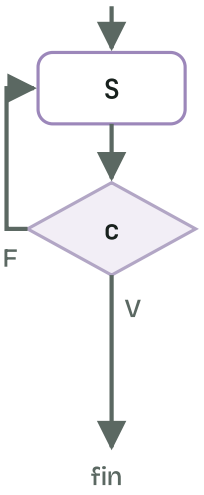
Cualquier composición iterativa se puede expresar con cualquier otra; hay que saber elegir la adecuada.



MIENTRAS

`MIENTRAS c HACER S`

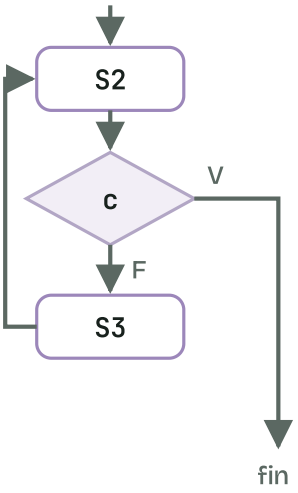
comprueba al principio: 0 o más vueltas; c es de continuación



REPETIR

`REPETIR S HASTA c`

comprueba al final: 1 o más vueltas; c es de parada



ITERAR

`ITERAR S2 DETENER c S3`

comprueba en medio: S2 al menos una vez; c es de parada

Dónde se comprueba la parada en cada composición. Al pasar de MIENTRAS a REPETIR o ITERAR, la condición se niega: una es de continuación y las otras, de parada.

Composición	Equivalente con…	Equivalente con…
`MIENTRAS c HACER S2`	`SI c ENTONCES REPETIR S2 HASTA NO c FIN_SI`	`ITERAR DETENER NO c; S2 FIN_ITERAR`
`REPETIR S2 HASTA c`	`S2; MIENTRAS NO c HACER S2`	`ITERAR S2 DETENER c FIN_ITERAR`
`ITERAR S2 DETENER c S3`	`S2; MIENTRAS NO c HACER S3; S2`	`S2; SI NO c ENTONCES REPETIR S3; S2 HASTA c FIN_SI`

> ⚠ **Ojo**
>
> Al pasar de MIENTRAS a REPETIR (o al revés) la condición se **niega**: una es de continuación y la otra de parada. Y el REPETIR necesita un SI delante porque el MIENTRAS puede no entrar nunca.

3.6 RECORRIENDO

`k RECORRIENDO [a, b] HACER S FIN_RECORRIENDO`: ejecuta S una vez por cada valor del intervalo, con k tomando a, a+1, ..., b. El inicio y la parada están **implícitos**. Si a > b el intervalo es vacío y S no se ejecuta. Variante: `EN SENTIDO INVERSO` (de b a a).

- S **no debe modificar** la variable de control k.
- No debe suponerse que los límites se reevalúan en cada vuelta; es sano que no cambien.
- Al terminar, el valor de k **no está definido** (como si no estuviera inicializada).

Producto de dos naturales sólo con sumas

```
LÉXICO
  a, b : Entero ≥ 0;    // datos
  p : Entero ≥ 0;       // resultado
  i : Entero ≥ 0;       // variable de la iteración
ALGORITMO
  Leer (a, b);
  p ← 0;                          // p = 0·a
  i RECORRIENDO [1, b] HACER
    p ← p + a                     // siguiente múltiplo de a
  FIN_RECORRIENDO;
  Escribir (p)                   // p = a·b
FIN.
```

Situación	Composición natural
Nº de vueltas conocido antes de empezar	RECORRIENDO
Puede no haber que entrar (comprobar antes)	MIENTRAS
Hay que hacer algo al menos una vez antes de comprobar	REPETIR
Leer / comprobar final / tratar	ITERAR

3.7 Problemas iterativos resueltos

Factorial

0! = 1 y n! = n · (n − 1)!. Tres versiones:

Versión 1 (MIENTRAS)

```
Leer (n);
fact ← 1; i ← 1;
MIENTRAS i < n HACER
  i ← i + 1;
  fact ← fact * i
FIN_MIENTRAS;
Escribir (fact)
```

Versión 3 (RECORRIENDO)

```
Leer (n);
fact ← 1;
i RECORRIENDO [2, n] HACER
  fact ← fact * i
FIN_RECORRIENDO;
Escribir (fact)
```

La de **RECORRIENDO**: el número de multiplicaciones se conoce antes de empezar (n − 1), así que la composición expresa directamente la idea y no hay que gestionar a mano el contador ni la condición. Además funciona sin casos especiales para n = 0 y n = 1 (el intervalo [2, n] es vacío y fact queda en 1).

Divisores propios y números primos

Divisores propios de a

```
Leer (a); n ← 0;
i RECORRIENDO [2, a - 1] HACER
  SI a MOD i = 0 ENTONCES
    Escribir (i); n ← n + 1
  FIN_SI
FIN_RECORRIENDO;
Escribir (n)
```

¿Es n primo? (búsqueda del primer divisor)

```
Leer (n);
SI n = 1 ENTONCES Escribir ("1 no es
primo")
SI_NO
  j ← 2;
  MIENTRAS n MOD j ≠ 0 HACER j ← j
+ 1 FIN_MIENTRAS;
  SI j = n ENTONCES Escribir (n, "
es primo")
  SI_NO Escribir (n, " no es primo")
FIN_SI
FIN_SI
```

- **Divisores**: ningún divisor propio supera a/2, así que basta `i RECORRIENDO [2, a DIV 2]`; mejor aún, recorrer hasta √a y anotar cada pareja i y a DIV i.

- **Primos**: es una búsqueda **con garantía de éxito** (n siempre se divide a sí mismo), por eso no hace falta comprobar el final. Mejoras de la práctica 5: descartar los pares (salvo el 2) y probar sólo divisores impares **hasta √n**: si no hay divisor por debajo de la raíz, tampoco por encima.
- Ojo: las transparencias escriben «1 es primo», pero el 1 **no** es primo por definición (sólo tiene un divisor).

Potencias de 2 en [x, y]

Solución 1: una función potencia

```
P2 : función (n : Entero) → Entero;
  LÉXICO p, i : Entero;
  ALGORITMO
    p ← 1;
    i RECORRIENDO [1, n] HACER p ←
p * 2 FIN_RECORRIENDO;
    P2 ← p
  FIN;
…
i RECORRIENDO [x, y] HACER Escribir
(P2 (i)) FIN_RECORRIENDO
```

Solución 2: cálculo incremental

```
Leer (x, y);
potencia2 ← 1;
i RECORRIENDO [1, x] HACER potencia2
← potencia2 * 2 FIN_RECORRIENDO;
{ e1 : potencia2 = 2^x }
i RECORRIENDO [x, y] HACER
  Escribir ("La potencia ", i, "-
ésima de 2 es ", potencia2);
  potencia2 ← potencia2 * 2
FIN_RECORRIENDO
```

La primera **recalcula cada potencia desde cero** (P2 (i) hace i multiplicaciones: en total del orden de $y^2 / 2$). La segunda **reutiliza la anterior**: cada término sale del previo con una sola multiplicación (del orden de y en total). Es la idea que reaparece en la serie de e^x de la práctica 5: calcular cada término a partir del anterior.

Parte entera de la raíz cuadrada

Se recorren los cuadrados 1, 4, 9, 16… sumando impares ($1 = 1$, $1 + 3 = 4$, $1 + 3 + 5 = 9$…: la suma de los k primeros impares es k^2).

Raíz entera por suma de impares

```
Leer (n);
rc ← 0; c ← 1; imp ← 1;
// en cada paso c = (rc + 1)²
MIENTRAS n ≥ c HACER
  imp ← imp + 2;
  c ← c + imp;
  rc ← rc + 1
FIN_MIENTRAS;
Escribir (rc)        // n = 10: c = 1, 4, 9, 16 → rc = 3
```

Fibonacci

$f_0 = f_1 = 1$ y $f_i = f_{i-1} + f_{i-2}$: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55... Se guardan los dos términos anteriores (ult y penult) y se desplazan en cada vuelta.

Término n-ésimo de Fibonacci

```
Leer (n);
SEGÚN n
  n < 2 : Escribir (1);
  n > 1 : penult ← 1; ult ← 1; fibo ← 2;
          i RECORRIENDO [3, n] HACER
            penult ← ult;
            ult ← fibo;
            fibo ← ult + penult
            { (fibo = fi) Y (ult = fi-1) Y (penult = fi-2) }
          FIN_RECORRIENDO;
          Escribir (fibo)
FIN_SEGÚN
```

Tema 4

4 Composición iterativa y noción de secuencia

La máquina secuencial, el invariante, los esquemas de recorrido y de búsqueda, cómo se combinan y los modelos de acceso secuencial. Incluye un simulador para ver los esquemas paso a paso.

4.1 Secuencias y máquina secuencial

Una **secuencia** es un conjunto de valores de cualquier tipo en el que **importa el orden**: [32, 64, 85] ≠ [85, 64, 32]. Su **longitud** (nº de elementos) **no se conoce a priori**; [ ] es la secuencia vacía (long = 0).

La **máquina secuencial** es una cinta con una **ventana** que deja ver un único elemento, el **elemento actual** (EA). A su izquierda queda la **parte izquierda** (Piz, lo ya tratado) y a su derecha la **parte derecha** (Pdr, lo pendiente). En las secuencias **marcadas**, tras el último elemento hay una **marca de fin** (MarcaFin, por ejemplo '#').



Piz: ya tratado EA: elemento actual Pdr: pendiente

La secuencia [32, 64, 85, 17] marcada con #, con la ventana sobre el 85.
Avanzar mueve la ventana una celda a la derecha.

Primitiva (primer modelo)	Efecto
`Comenzar (S)`	Inicia la consulta; el **primer elemento** pasa a ser el EA
`Avanzar (S)`	Avanza al siguiente elemento
`EA (S)`	Devuelve el elemento actual (MarcaFin al final)
`Crear (S)`	Crea S vacía para añadir elementos por la derecha
`Registrar (S, e)`	Graba e como último elemento
`Marcar (S)`	Graba la marca de fin

En las secuencias **no marcadas** se usa una función booleana `FDS (S)` (fin de secuencia), verdadera cuando se ha sobrepasado el último elemento.

> **En C++ (práctica 6)**
>
> La entrada estándar hace de secuencia del primer modelo: `cin >> EA` antes del bucle es **Comenzar**, `while (EA ≠ MF)` la comprobación de la marca y otra lectura al final del cuerpo es **Avanzar**.

4.2 Análisis iterativo y el invariante

Una iteración se modela como el **tratamiento de una secuencia**. Los estados intermedios no son iguales, pero todos comparten una propiedad que se cumple en cada paso: el **invariante**. Al terminar, invariante + condición de parada ⇒ postcondición.

Longitud de un texto

```
Léxico
  conta : Entero ≥ 0;
  S : secuencia de Carácter;
Algoritmo
  Comenzar (S); conta ← 0;                // inicialización
  MIENTRAS EA (S) ≠ MarcaFin HACER
    conta ← conta + 1; Avanzar (S)        // cuerpo
  FIN_MIENTRAS;
  Escribir (conta)                         // finalización
Fin.
// Inv: conta = número de elementos de Piz (S)
```

Cómo se obtiene cada parte (razonamiento inductivo)

- **Inicialización**: para cada variable v, ¿cuál es la solución para la **secuencia vacía**? (casos base: `conta ← 0`, `suma ← 0`).
- **Tratamiento elemental**: suponemos conocida la solución para Piz, guardada en v (**hipótesis de inducción**); ¿qué hay que hacer para tener la de Piz más el EA? `v ← g (v, EA)`: `suma ← suma + EA (S)`, `numElem ← numElem + 1`.

Ejemplo: media de una secuencia de notas. Post: { suma = ΣS_i, numElem = long (S) }. Al final, si numElem > 0 se escribe suma / numElem; si no, «Secuencia vacía».

4.3 Esquemas de recorrido

Un **esquema algorítmico** es una plantilla aplicable a una **clase de problemas**. **Recorrido** (enumeración secuencial): aplicar el mismo tratamiento a **todos** los elementos. Hay que decidir si:

- **H1**: la **secuencia vacía** se puede tratar como las demás.
- **H2**: el **primer elemento** se puede tratar como los demás.

Esquema 1 · H1 y H2

```
Comenzar;
{ Inicialización del tratamiento }
MIENTRAS EA ≠ MarcaFin HACER
  { Tratamiento de EA }
  Avanzar
FIN_MIENTRAS;
{ Terminación del tratamiento }
```

Esquema 2 · ¬H1 y H2

```
Comenzar;
SEGÚN EA
  EA = MarcaFin : { Tratamiento sec. vacía }
  EA ≠ MarcaFin :
    { Inicialización del tratamiento }
    REPETIR
      { Tratamiento de EA }
      Avanzar
    HASTA EA = MarcaFin;
    { Terminación del tratamiento }
FIN_SEGÚN
```

Esquema 3 · ¬H1 y ¬H2

```
Comenzar;
SEGÚN EA
  EA = MarcaFin : { Tratamiento sec. vacía }
  EA ≠ MarcaFin :
    { Tratamiento del 1er elemento }      // casos base: secuencia de un elemento
    ITERAR
      Avanzar
      DETENER EA = MarcaFin;
      { Tratamiento de EA }
    FIN_ITERAR;
    { Terminación del tratamiento }
FIN_SEGÚN
```

Con la **media**: esquema 1 (sum ← 0; n ← 0; al final SEGÚN n = 0 / n > 0); esquema 2 (la secuencia vacía se trata al principio y la división se hace sin comprobar); esquema 3 (sum ← EA (S); n ← 1 como inicialización).

4.4 Problemas de recorrido

Meseta mayor

Meseta = sucesión de valores idénticos. En S = [35, 18, 18, 18, 5, 62, 35, 35, 11, 11, 11, 11, 9, 9] la mayor mide **4** (los 11). Inv: { p = LongMesetaMayor (Piz), u = último (Piz), m = LongÚltMeseta (Piz) }. **u no está definido para la secuencia vacía** (¬H2), así que hay dos opciones:

Con H2 (esquema 1), enteros > 0

```
Comenzar (S);
p ← 0; m ← 0; u ← 0;   // «truco sucio»: 0 no aparece
MIENTRAS EA (S) ≠ MarcaFin HACER
  SI EA (S) = u ENTONCES m ← m + 1
  SI_NO m ← 1 FIN_SI;
  SI m > p ENTONCES p ← m FIN_SI;
  u ← EA (S);
  Avanzar (S)
FIN_MIENTRAS;
Escribir (p)
```

Con ¬H2 (esquema 3)

```
Comenzar (S);
SEGÚN EA (S)
  EA (S) = MarcaFin : Escribir ("Secuencia vacía");
  EA (S) ≠ MarcaFin :
    p ← 1; m ← 1; u ← EA (S);
    ITERAR
      Avanzar (S)
      DETENER EA (S) = MarcaFin;
      SI EA (S) = u ENTONCES m ← m + 1 SI_NO m ← 1 FIN_SI;
      SI m > p ENTONCES p ← m FIN_SI;
      u ← EA (S)
    FIN_ITERAR;
    Escribir (p)
FIN_SEGÚN
```

Máximo

max no está definido para la vacía: esquema 3 con `m ← EA (S)` al principio y `SI EA (S) > m ENTONCES m ← EA (S)` en el cuerpo.

Copia de una secuencia

Recorrido de S1 con creación simultánea de S2

```
Comenzar (S1); Crear (S2);
// Inv: Piz (S1) = Piz (S2)
MIENTRAS EA (S1) ≠ MarcaFin HACER
  Registrar (S2, EA (S1));
  Avanzar (S1)
FIN_MIENTRAS;
Marcar (S2)
```

Máximos y mínimos con posiciones

Para S = [34, 1, 90, 39, 1, 15, 90, 25, 1, 10]: **ma = 90, pma = 3, uma = 7, nma = 2** y **mi = 1, pmi = 2, umi = 9, nmi = 3**. Las variables dependen unas de otras (pma, uma, nma de ma y de pos), así que el orden de las actualizaciones importa: primero pos, luego un SEGÚN con ma.

Inicialización y cuerpo

```
// inicialización (esquema 3, primer elemento)
pos ← 1; ma ← EA (S); pma ← 1; uma ← 1; nma ← 1;
         mi ← EA (S); pmi ← 1; umi ← 1; nmi ← 1;
// cuerpo
pos ← pos + 1;
SEGÚN EA (S), ma
  EA (S) = ma : uma ← pos; nma ← nma + 1;
  EA (S) > ma : uma ← pos; pma ← pos; ma ← EA (S); nma ← 1;
  EA (S) < ma : ;
FIN_SEGÚN;
SEGÚN EA (S), mi      // simétrico, con < y >
  …
FIN_SEGÚN
```

Intercalación de secuencias ordenadas

S1 = [25, 11, 11, 3, 1, 1] y S2 = [38, 25, 14, 11, 9, 9, 7], decrecientes → R = [38, 25, 25, 14, 11, 11, 11, 9, 9, 7, 3, 1, 1]. Inv: Piz (R) = Intercalar (Piz (S1), Piz (S2)). En cada paso se registra el **mayor** de los dos EA (o los dos si son iguales) y se avanza en esa secuencia; cuando una se acaba se copia el resto de la otra con la acción `CopiarResto`.

Intercalar

```
Comenzar (S1); Comenzar (S2); Crear (R);
MIENTRAS (EA (S1) ≠ MarcaFin) Y (EA (S2) ≠ MarcaFin) HACER
  SEGÚN EA (S1), EA (S2)
    EA (S1) = EA (S2) : Registrar (R, EA (S1)); Registrar (R, EA (S2));
                        Avanzar (S1); Avanzar (S2);
    EA (S1) > EA (S2) : Registrar (R, EA (S1)); Avanzar (S1);
    EA (S1) < EA (S2) : Registrar (R, EA (S2)); Avanzar (S2);
  FIN_SEGÚN
FIN_MIENTRAS;
SEGÚN EA (S1), EA (S2)
  (EA (S1) ≠ MarcaFin) Y (EA (S2) = MarcaFin) : CopiarResto (S1, R);
  (EA (S1) = MarcaFin) Y (EA (S2) ≠ MarcaFin) : CopiarResto (S2, R);
  EN_OTRO_CASO : ;
FIN_SEGÚN;
Marcar (R)
```

4.5 Esquema de búsqueda

Búsqueda: encontrar el **primer** elemento que cumple una propiedad Pro (no hace falta recorrer todo).

- Post: { (Pro (EA) = Verdadero O EA = MarcaFin) Y ningún e de Piz cumple Pro }
- Inv: { ningún elemento de Piz cumple Pro }
- Condición de continuación: `(EA ≠ MarcaFin) Y NO Pro (EA)`

Esquema de búsqueda

```
Comenzar;  { Establecimiento inicial del invariante }
MIENTRAS (EA ≠ MarcaFin) Y NO Pro (EA) HACER
  { Tratamientos para mantener el invariante }   // si Pro depende de Piz
  Avanzar
FIN_MIENTRAS;
SEGÚN EA
  EA = MarcaFin : { Tratamiento: no encontrado }
  EA ≠ MarcaFin : { Tratamiento: EA encontrado }
FIN_SEGÚN
```

> ⚠ **YDESPUÉS**
>
> Si es erróneo consultar Pro en la marca (o no hay marca y se usa FDS), se usa el **Y condicional**: `MIENTRAS (EA ≠ MarcaFin) YDESPUÉS NO Pro (EA)` sólo evalúa la segunda parte si la primera es verdadera. En C++, `&&` ya funciona así.

Al menos x apariciones de c: Pro depende de Piz (la x-ésima c). Inv: { noc = NroOcur (c, Piz) Y noc < x }.

Búsqueda con tratamiento

```
Comenzar (S); Leer (x, c); noc ← 0;
MIENTRAS (EA (S) ≠ MarcaFin) Y ((EA (S) ≠ c) O (noc ≠ x - 1)) HACER
  SI EA (S) = c ENTONCES noc ← noc + 1 FIN_SI;   // mantener el invariante
  Avanzar (S)
FIN_MIENTRAS;
SEGÚN EA (S)
  EA (S) = MarcaFin : Escribir (c, " no aparece ", x, " veces");
  EA (S) ≠ MarcaFin : Escribir (c, " aparece al menos ", x, " veces")
FIN_SEGÚN
```

Búsqueda con garantía de éxito: si se sabe que algún elemento cumple Pro, se omite la comparación con la marca. Ejemplo: *¿un texto es blanco?* `MIENTRAS EA (S) = ' ' HACER Avanzar (S)` (la marca no es un blanco, así que para seguro).

Longitud de la primera palabra: doble búsqueda; la del comienzo (saltar blancos, con garantía de éxito) y la del final, contando: `lpp ← 0; MIENTRAS (EA (S) ≠ MarcaFin) Y (EA (S) ≠ ' ') HACER lpp ← lpp + 1; Avanzar (S)`.

Sí: cuando empieza, ya sabemos que EA no es la marca ni un blanco (hay palabra), así que el cuerpo se ejecuta al menos una vez y `REPETIR lpp ← lpp + 1; Avanzar (S) HASTA (EA (S) = MarcaFin) O (EA (S) = ' ')` es correcto y expresa esa certeza. El MIENTRAS también es correcto y sigue el esquema general de búsqueda.
¿Contiene 'A', 'E' e 'I' en este orden? Tres búsquedas iguales: se define una acción (las secuencias se pasan como **dato–resultado**, porque avanzar cambia su estado interno).

Tres búsquedas en secuencia

```
BuscarLetra : acción (dato c : Carácter; dato-resultado S : secuencia de
Carácter)
  Pre : { Estado (S) = consulta }
  Post : { (EA (S) = c) O (EA (S) = MarcaFin) }
  ALGORITMO
    MIENTRAS (EA (S) ≠ MarcaFin) Y (EA (S) ≠ c) HACER Avanzar (S) FIN_MIENTRAS
  FIN;
…
Comenzar (S);
BuscarLetra ('A', S);
SEGÚN EA (S)
  EA (S) = MarcaFin : Escribir ("No");
  EA (S) ≠ MarcaFin : BuscarLetra ('E', S);  … (igual con 'I')
FIN_SEGÚN
```

- **¿Sería correcto llamar a BuscarLetra tres veces seguidas y comprobar sólo al final?** Sí: si una búsqueda llega a la marca, las siguientes no avanzan (su condición es falsa) y al final EA = MarcaFin → «No».

- **¿Y para 'A', 'A', 'E'?** Ninguna de las dos versiones sirve: tras encontrar la primera 'A', la segunda búsqueda de 'A' la encuentra en el mismo sitio sin avanzar.
- **¿Cómo arreglarlo en el algoritmo principal?** Haciendo `Avanzar (S)` después de cada búsqueda con éxito, antes de la siguiente.
- **¿Y modificando la acción?** Empezando con un Avanzar; entonces cambia su especificación: Pre { EA (S) ≠ MarcaFin } y la Post habla del primer c *a la derecha* del EA inicial (la primera búsqueda tendría que hacerse aparte o con otra acción).

Igualdad de dos secuencias: búsqueda de la primera diferencia recorriéndolas a la vez (Inv: Piz (S1) = Piz (S2)); son iguales si las dos acaban en la marca a la vez.

4.6 Combinación de recorrido y búsqueda

Hay que identificar la naturaleza de cada subproblema y cómo se combinan: **anidado** (uno forma parte del tratamiento del EA del otro) o **en secuencia** (uno detrás del otro).

Anidados: elementos de S1 que están en S2 (ambas decrecientes, sin repetidos)

```
Comenzar (S1); Comenzar (S2);
MIENTRAS EA (S1) ≠ MarcaFin HACER
// 1) recorrido de S1
  MIENTRAS (EA (S2) ≠ MarcaFin) Y
(EA (S2) > EA (S1)) HACER
    Avanzar (S2)
// 2) búsqueda en S2
  FIN_MIENTRAS;
  SI (EA (S2) ≠ MarcaFin) Y (EA (S2)
= EA (S1)) ENTONCES
    Escribir (EA (S1), " está
también en S2")
  SI_NO Escribir (EA (S1), " no está
en S2") FIN_SI;
  Avanzar (S1)
FIN_MIENTRAS
```

En secuencia: ¿hay en S2 un elemento igual a la suma de S1?

```
Comenzar (S1);
SEGÚN EA (S1)
  EA (S1) = MarcaFin : Escribir
("Secuencia S1 vacía");
  EA (S1) ≠ MarcaFin :
    suma ← EA (S1);
    ITERAR
// 1) recorrido de S1 (¬H1, ¬H2)
      Avanzar (S1)
      DETENER EA (S1) = MarcaFin;
      suma ← suma + EA (S1)
    FIN_ITERAR;
    Comenzar (S2);
// 2) búsqueda en S2, en secuencia
    MIENTRAS (EA (S2) ≠ MarcaFin) Y
(EA (S2) ≠ suma) HACER
      Avanzar (S2)
    FIN_MIENTRAS;
    SI EA (S2) = MarcaFin ENTONCES
Escribir ("No") SI_NO Escribir
("Sí") FIN_SI
FIN_SEGÚN
```

---

a) S1 = [30, 24, 12, 6, 5], S2 = [60, 58, 30, 15, 12, 3, 2]: 30 **sí** (S2 avanza 60, 58 y para en 30); 24 no (para en 15); 12 **sí**; 6 no (para en 3); 5 no.

b) S1 = [60, 58, 30, 15, 12, 3, 2], S2 = [30, 24, 12, 6, 5]: 60 no y 58 no (S2 está en 30, que no es mayor); 30 **sí**; 15 no (S2 avanza a 12); 12 **sí**; 3 no (S2 llega a la marca); 2 no.

Como S2 está ordenada, cada búsqueda continúa donde quedó la anterior: S2 se recorre una sola vez en total.

4.7 Modelos de acceso secuencial

Dos características: si el primer elemento está **disponible** nada más iniciar (**DP**) o hace falta avanzar antes (¬DP), y si el final se detecta al **sobrepasar** el último elemento (**FS**) o estando sobre él (¬FS).

	DP	¬DP
FS	**Primer modelo** (secuencias marcadas, FDS)	Cuarto modelo
¬FS	Tercer modelo (búsquedas en tablas)	**Segundo modelo**

Segundo modelo. Primitivas: `Iniciar (S)` (el EA aún no es significativo), `Avanzar (S)`, `EA (S)`, `EsVacía (S)` y `EsÚltimo (S)` (ya no se puede avanzar). Como hay que avanzar antes de tratar y al detectar el final el EA todavía hay que tratarlo, **cambia el orden** de los tratamientos.

Segundo modelo · esquema 1 (H1 y H2)

```
Iniciar;
{ Inicialización del tratamiento }
MIENTRAS NO EsÚltimo HACER
  Avanzar;
  { Tratamiento de EA }
FIN_MIENTRAS;
{ Terminación del tratamiento }
```

Segundo modelo · búsqueda

```
Iniciar;
SEGÚN EsVacía
  EsVacía : { Tratamiento: no
encontrado }
  NO EsVacía :
    REPETIR
      Avanzar        // y
tratamientos para mantener el Inv
    HASTA EsÚltimo O Pro (EA);
    SEGÚN EA
      Pro (EA) : { Tratamiento: EA
encontrado }
      NO Pro (EA) : { Tratamiento:
no encontrado }
    FIN_SEGÚN
FIN_SEGÚN
```

En el **primer modelo con FDS** los esquemas son los mismos cambiando `EA = MarcaFin` por `FDS`; en la búsqueda es obligatorio `MIENTRAS NO FDS YDESPUÉS NO Pro (EA)` para no consultar un EA que ya no existe.

4.8 Simulador: los esquemas paso a paso

Elige un algoritmo, escribe la cinta (lo que haya tras el primer # se ignora) y pulsa **Paso**. Verás qué línea se ejecuta, cómo avanza la ventana, cómo crece la parte izquierda y que el **invariante** se cumple antes de cada comprobación de la condición.

A

Anexo

Chuletas para repasar

La traducción de la notación SP a C++ en una tabla, los esquemas para memorizar y el acceso a los cuestionarios de Wooclap de cada tema.

A.1 De la notación SP a C++

Notación SP	C++
`Entero` · `Real` · `Booleano` · `Carácter`	`int` (short, long) · `float`, `double` · `bool` · `char` (y `string` para textos)
`Verdadero` / `Falso`	`true` / `false`
`x ← e`	`x = e;`
`=` · `≠` · `≤` · `≥`	`==` · `!=` · `<=` · `>=`
`DIV` · `MOD` · `/` (real)	`/` entre enteros · `%` · `/` con algún real (`float (a) / b`)
`Y` · `O` · `NO`	`&&` · `||` · `!` (o `not`)
`Car (e)` · `Ord (c)` · `Sucesor (c)`	`char (e)` · `int (c)` · `char (int (c) + 1)`
`Leer (a, b)` · `Escribir ("x =", x)`	`cin >> a >> b;` · `cout << "x =" << x << endl;`
`c : constante = valor`	`const int C = valor;`
`T = TIPO < c1: T1; c2: T2 >`	`struct T { T1 c1; T2 c2; };`
`SI c ENTONCES S SI_NO R FIN_SI`	`if (c) { S } else { R }`
`SEGÚN` con condiciones	`if … else if … else` (o `switch` si es una expresión ordinal frente a constantes)
`MIENTRAS c HACER S`	`while (c) { S }`
`REPETIR S HASTA c`	`do { S } while (!(c));`
`ITERAR S1 DETENER c S2`	`S1; while (!(c)) { S2; S1; }` o `do { S1; if (c) break; S2; } while (true);`

Notación SP	C++
`i RECORRIENDO [a, b]` (inverso)	`for (i = a; i ⩽ b; i = i + 1)` (`for (i = b; i ⩾ a; i = i - 1)`)
`A : acción (dato x: T; dato-resultado y: T)`	`void A (T x, T &y)`
`f : función (x: T) → R` · `f ← valor`	`R f (T x)` · `return valor;`

A.2 Esquemas para memorizar

Problema	Esquema	Clave
Tratar todos los elementos; la vacía y el primero como los demás	Recorrido 1 (MIENTRAS)	Inicialización = solución de la vacía
La vacía es especial; el primero no	Recorrido 2 (SEGÚN + REPETIR)	Media sin dividir entre 0
La vacía y el primero son especiales	Recorrido 3 (SEGÚN + ITERAR)	Máximo, mínimo, meseta, pares adyacentes
Encontrar el primero que cumple Pro	Búsqueda	`(EA ≠ MF) Y NO Pro (EA)`; al final, SEGÚN EA
Pro depende de lo ya visto	Búsqueda con tratamientos	Mantener en la iteración las variables del invariante
Se sabe que existe	Búsqueda con garantía de éxito	No hace falta comparar con la marca
Tratar todo y, para cada elemento, buscar en otra	Búsqueda anidada en un recorrido	Si están ordenadas, la búsqueda no recomienza
Calcular algo y luego buscarlo	Recorrido y búsqueda en secuencia	Suma de S1 en S2

A.3 Wooclap: cuestionarios a tu ritmo

Los cuestionarios de clase siguen abiertos para hacerlos a tu ritmo. Entra en wooclap.com con el código de tu grupo o abre directamente el cuestionario:

Tema	Grupo 2	Grupo 3	Grupo 4
Tema 0	`TRBDNKN` · cuestionario ↗	`HHGTJYZ` · cuestionario ↗	`LGXZNGU` · cuestionario ↗
Tema 1	`ZKDMPGA` · cuestionario ↗	`URFAGXT` · cuestionario ↗	`AONIYYM` · cuestionario ↗
Tema 2	`ELMPMHL` · cuestionario ↗	`ERBSTMQ` · cuestionario ↗	`TGJBPSI` · cuestionario ↗
Tema 3	`PNHYNRQ` · cuestionario ↗	—	—

Para practicar sin conexión tienes el autotest de IP.

Basado en las transparencias de teoría de los temas 0 a 4 de la asignatura (las erratas de las transparencias están corregidas). El código de los ejemplos en C++ está compilado y probado.